MASKAGLIA: A New, Efficient Approach to Masked Discrete Gaussian Sampling

Calvin Abou Haidar Thomas Espitau
Clément Hoffmann Mehdi Tibouchi

NTT, PQ Shield

Friday 8th May, 2026



Gaussian Distributions: From $\mathbb{R}$ to $\mathbb{Z}$

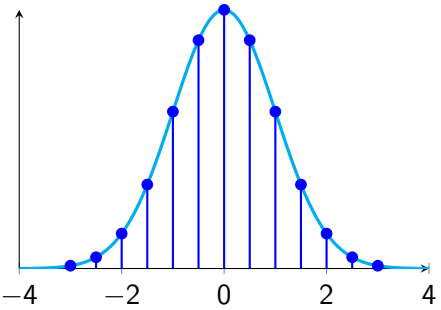


Continuous Gaussian $\mathcal{N}(c, \sigma^2)$

- Defined over $\mathbb{R}$.
- Density: $f(x) \propto \exp\left(-\frac{(x-c)^2}{2\sigma^2}\right)$.
- Continuous "bell curve".

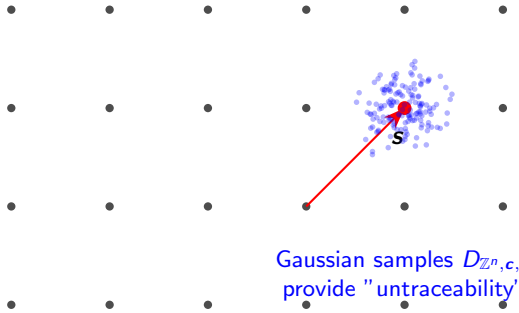
Discrete Gaussian $D_{\mathbb{Z},c,\sigma}$

- Defined over $\mathbb{Z}$.
- Mass: $P(X = x) \propto \exp\left(-\frac{(x-c)^2}{2\sigma^2}\right)$.
- Essential for lattice-based primitives.



Role in Post-Quantum Cryptography

- **Lattice Security**: Fundamental for security proofs (LWE, SIS).
- **Signatures**: Core component of *Hash-and-Sign* schemes (Falcon, Mitaka, HAWK).
- **Geometric Erasure**: Gaussian noise "smears" the lattice to hide the secret key structure.



Gaussian samples $D_{\mathbb{Z}^n,\boldsymbol{c},\sigma}$ provide "untraceability"

A Historical Implementation Challenge



Implementing secure Gaussian samplers is notoriously difficult:

- **Precision** Handling transcendental functions (exp, ln) and real numbers without introducing bias.
- **Timing** Rejection sampling or inversion-based methods often leak data through execution time.
- **Masking** Expensive

Scheme designers and implementers avoid Gaussian sampling as much as possible (heurstic error distribution).

Introduction to Boolean Masking



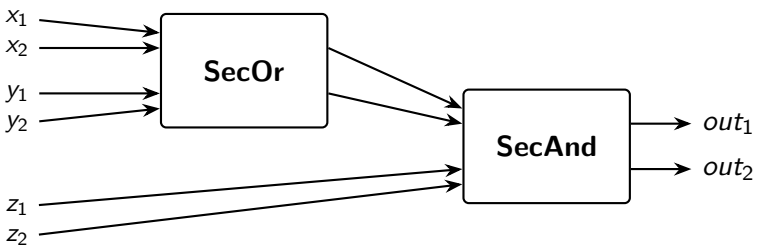
Principle: The sensitive variable x is split into d random shares:

$$x = x_1 \oplus x_2 \oplus \cdots \oplus x_d$$

Introduction to Boolean Masking

Principle: The sensitive variable x is split into d random shares:

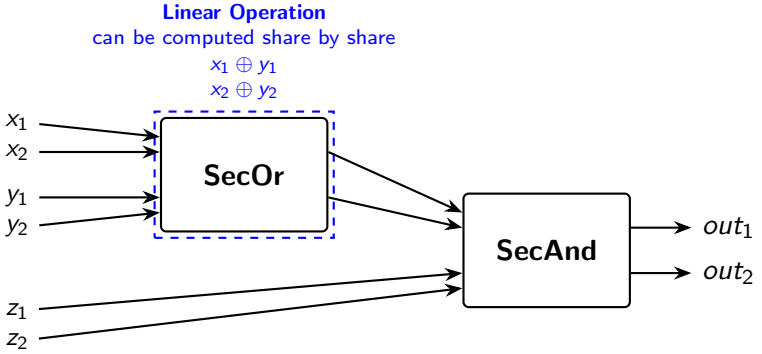
$$x = x_1 \oplus x_2 \oplus \cdots \oplus x_d$$



Introduction to Boolean Masking

Principle: The sensitive variable x is split into d random shares:

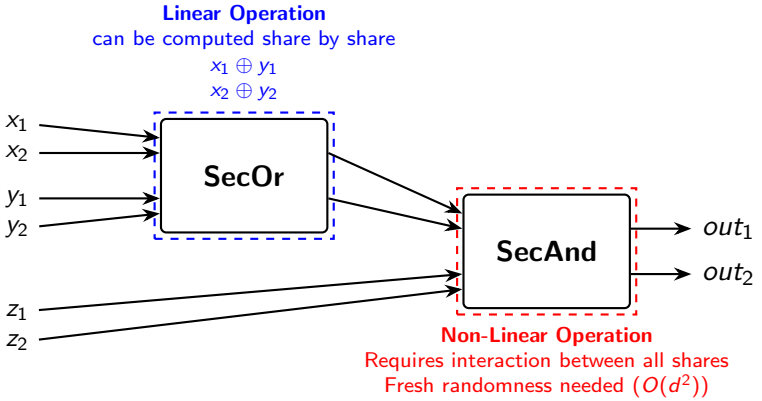
$$x = x_1 \oplus x_2 \oplus \cdots \oplus x_d$$



Introduction to Boolean Masking

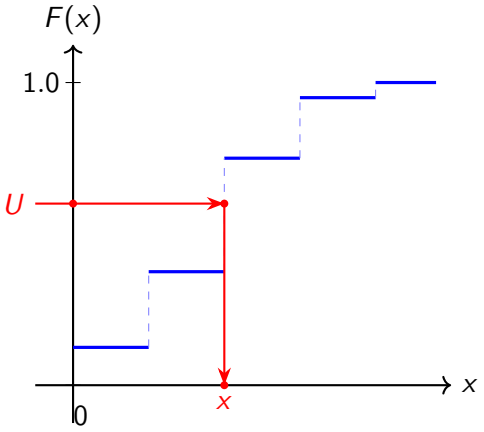
Principle: The sensitive variable x is split into d random shares:

$$x = x_1 \oplus x_2 \oplus \cdots \oplus x_d$$



Inverse Transform Sampling

- If $U \sim \mathrm{Unif}([0,1])$, then $F^{-1}(U)$ has CDF F.
- For discrete distributions (like our Gaussian), $F(x)$ is a step function.

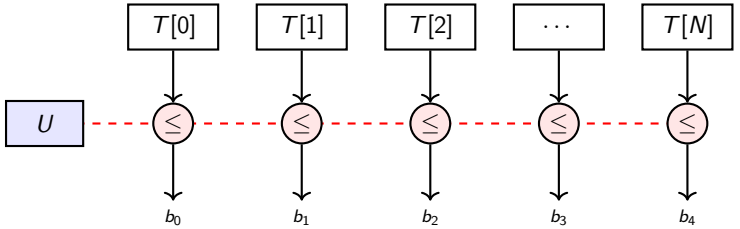


CDT Sampling

Cumulative Distribution Table (CDT)

- Precompute a table T containing the step boundaries: $T[x] = P(X \leq x)$.
- Draw a uniform random value U.
- Find the smallest index x such that $U \leq T[x]$.

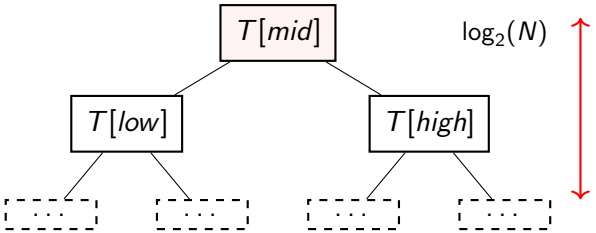
Constant-Time Constraint: We must evaluate the comparison $U \leq T[x]$ against **every single element** in the table.



Optimizing CDT: Binary Search Trees

To reduce the number of comparisons, one can organize the CDT as a binary tree.

- Instead of N comparisons, we only need $\lceil \log_2(N) \rceil$.
- **Masked Challenge**: The path taken in the tree must remain secret.



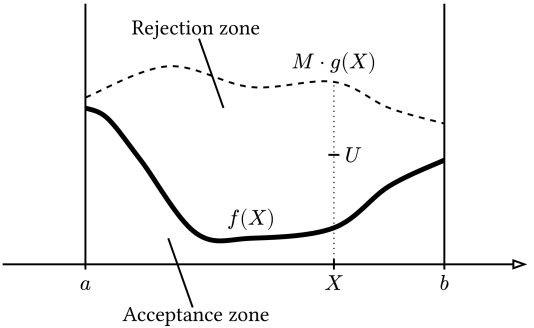
State of the Art (Eid et al., CHES26)

Rejection Sampling

- Target distribution: $f(X)$ (hard to sample directly).
- Proposal distribution: $g(X)$ (easy to sample).
- Find a constant M such that $M \cdot g(X) \geq f(X)$ everywhere.

The Algorithm

1. Draw $X \sim g(X)$.
2. Draw $U \sim \text{Unif}(0, M \cdot g(X))$.
3. If $U \leq f(X)$, **Accept** X.
4. Else, **Reject** and restart.



Presentation Outline



1. From continuous to discrete
2. Moving everything to integers
3. Efficient gadgets
4. Performance and applications

Presentation Outline



1. From continuous to discrete
2. Moving everything to integers
3. Efficient gadgets
4. Performance and applications

Initial sampler



Alg. 1 :: Continuous Gaussian Sampler (Marsaglia, 1960s)

Output: $Z \sim \mathcal{N}(0, \sigma^2)$

repeat
  $X \leftarrow \text{Exp}(\lambda)$
  $Y \leftarrow \text{Exp}(\lambda)$
  $\epsilon \leftarrow \{\pm 1\}$
until $2\lambda X \geq (Y - \lambda)^2$
return $\epsilon \cdot Y$

Unified Sampling: Continuous and Discrete

The rejection template is identical for both supports $\mathcal{S} \in \{\mathbb{R}, \mathbb{Z}\}$, differing only in the nature of the Laplace proposal.

Alg. 2 :: Continuous Gaussian

repeat
  $X \leftarrow \text{Exp}(\lambda)$
  $Y \leftarrow \text{Lap}_c(\lambda)$
until $X \geq (|Y - c|/s - s/2)^2$
return Y

Alg. 3 :: Discrete Gaussian

repeat
  $X \leftarrow \text{Exp}(\lambda)$
  $Y \leftarrow DL_c(\lambda)$
until $X \geq (|Y - c|/s - s/2)^2$
return Y

Setting the Parameters: The $\ln 2$ Choice



Impact on the Standard Deviation

The relationship between the scaling factor s and the standard deviation σ is:

$$\sigma = \frac{s}{\sqrt{2\lambda}}$$

$$\lambda = \ln 2$$

Setting the Parameters: The $\ln 2$ Choice



Impact on the Standard Deviation

The relationship between the scaling factor s and the standard deviation σ is:

$$\sigma = \frac{s}{\sqrt{2\lambda}}$$

$$\lambda = \ln 2$$

Why this specific value?

- **High Efficiency**: Good rejection factor
- **Masking-Friendly**: $\exp(-\ln 2 \cdot k) = 2^{-k}$ can be expressed with simple bit manipulations and Bernoullis.

Sampling the Proposal: Discrete Laplace



Alg. 4 :: Discrete Laplace $DL_c(\ln 2)$

Input : Center $c \in \mathbb{R}$
Output: $Y \sim DL_c(\ln 2)$

$n \leftarrow \lfloor c \rceil; \quad \delta \leftarrow c - n$
$G \leftarrow \text{Geom}(1/2)$
$S \leftarrow \text{Bern}\left(\frac{2^{\delta}}{2^{\delta}+2^{1-\delta}}\right)$
if $S = 1$ **then return** $n + (G + 1)$
else return $n - G$

Sampling the Proposal: Discrete Laplace



Alg. 4 :: Discrete Laplace $DL_c(\ln 2)$

Input : Center $c \in \mathbb{R}$
Output: $Y \sim DL_c(\ln 2)$

1 $n \leftarrow \lfloor c \rceil$; $\quad \delta \leftarrow c - n$
2 $G \leftarrow \text{Geom}(1/2)$ // Easy to mask
3 $S \leftarrow \text{Bern}\left(\frac{2^\delta}{2^\delta + 2^{1-\delta}}\right)$ // Easy to mask for some parameters
4 **if** $S = 1$ **then return** $n + (G + 1)$
5 **else return** $n - G$

Sampling the Proposal: Exponential



Alg. 5 :: Exponential sampler for $\lambda = \ln 2$

Output: $X \sim \mathrm{Exp}(\ln 2)$

1 $K \leftarrow \mathrm{Geom}(1/2)$
2 $V \leftarrow \mathrm{Unif}([1/2, 1])$
3 $U \leftarrow -\frac{\ln V}{\ln 2}$
4 **return** $K + U$

Sampling the Proposal: Exponential



Alg. 5 :: Exponential sampler for $\lambda = \ln 2$

Output: $X \sim \text{Exp}(\ln 2)$

1 $K \leftarrow \text{Geom}(1/2)$ `// Easy to mask`

2 $V \leftarrow \text{Unif}([1/2, 1])$

3 $U \leftarrow -\frac{\ln V}{\ln 2}$ `// Transcendental: The bottleneck!`

4 **return** $K + U$

Presentation Outline



1. From continuous to discrete
2. Moving everything to integers
3. Efficient gadgets
4. Performance and applications

Rational Parameters and Integer Rejection



Assume the scaling factor s and center c are rational numbers:

$$s = \frac{p}{q} \in \mathbb{Q}_{>0}, \quad c = \frac{p'}{q'} \in \mathbb{Q} \quad \text{with } p, q, p', q' \in \mathbb{Z}$$

Rational Parameters and Integer Rejection

Assume the scaling factor s and center c are rational numbers:

$$s = \frac{p}{q} \in \mathbb{Q}_{>0}, \quad c = \frac{p'}{q'} \in \mathbb{Q} \quad \text{with } p, q, p', q' \in \mathbb{Z}$$

Let $g = \gcd(2q^2, p^2 q')$. We define the following integer constants:

$$L_y = \frac{2q^2|yq' - p'| - p^2 q'}{g} \qquad K_0 = \frac{(2pqq')^2}{g^2}$$

Conclusion

The rejection condition $X = K + U \geq (|Y - c|/s - s/2)^2$ exactly becomes

$$K_0 \cdot (K + U) < L_y^2$$

Rational Parameters and Integer Rejection

Assume the scaling factor s and center c are rational numbers:

$$s = \frac{p}{q} \in \mathbb{Q}_{>0}, \quad c = \frac{p'}{q'} \in \mathbb{Q} \quad \text{with } p, q, p', q' \in \mathbb{Z}$$

Let $g = \gcd(2q^2, p^2 q')$. We define the following integer constants:

$$L_y = \frac{2q^2|yq' - p'| - p^2 q'}{g} \qquad K_0 = \frac{(2pqq')^2}{g^2}$$

Conclusion

The rejection condition $X = K + U \geq (|Y - c|/s - s/2)^2$ exactly becomes

$$K_0 \cdot (K + U) < L_y^2$$

The only non-integer value is $U \cdot K_0 \in [0, K_0[$

Replacing U



Defining the Integer Proxy W

We define the integer-valued random variable:

$$W = \lfloor K_0 U \rfloor \in \{0, 1, \ldots, K_0 - 1\}$$

Replacing U



Defining the Integer Proxy W

We define the integer-valued random variable:

$$W = \lfloor K_0 U \rfloor \in \{0, 1, \ldots, K_0 - 1\}$$

For a fixed y, the test $K_0 K + W < L_y^2$ becomes:

- **Always True** if $K < L_y^2/K_0$ (Early reject)
- **Always False** if $K > L_y^2/K_0$ (Accept)
- **Boundary Case**: Test $W < L_y^2 \pmod{K_0}$

The Integer Proxy Distribution



Thresholds and Quadratic Residues

Since $R_y = L_y^2 \pmod{K_0}$, the set of all possible thresholds $\mathcal{R}_{s,c}$ is finite and determined by the **quadratic residues** modulo K_0.

- We only need to evaluate $W < R_y$ for $R_y \in \mathcal{R}_{s,c}$.

The Integer Proxy Distribution



Thresholds and Quadratic Residues

Since $R_y = L_y^2 \pmod{K_0}$, the set of all possible thresholds $\mathcal{R}_{s,c}$ is finite and determined by the **quadratic residues** modulo K_0.

- We only need to evaluate $W < R_y$ for $R_y \in \mathcal{R}_{s,c}$.

Exact CDF Matching

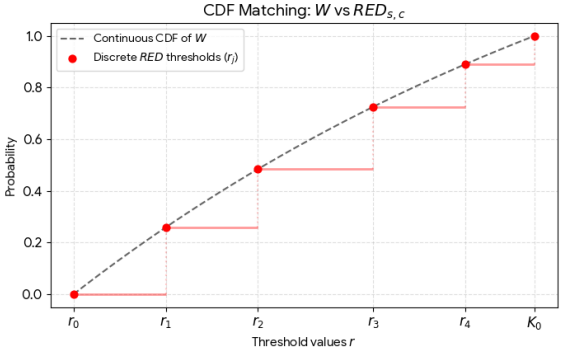
For $r \in \{0, \ldots, K_0\}$, $Pr[W < r] = Pr\left[U < \frac{r}{K_0}\right] = 2(1 - 2^{-r/K_0}) = 2 - 2^{1-r/K_0}$

The Rejection-Equivalent Distribution (RED)

Definition and CDF Matching

$RED_{s,c}$ is a discrete distribution supported on the finite set of thresholds

$$Pr[RED < r_j] = Pr[W < r_j] = 2 - 2^{1-r_j/K_0}$$



The Final Integer Sampler



Alg. 6 :: Integer Gaussian Sampler

Input : Rational $s = p/q > 0$ and $c = p'/q'$
Output: $Z \sim D_{\mathbb{Z},c,\sigma}$ where $\sigma = s/\sqrt{2\ln 2}$

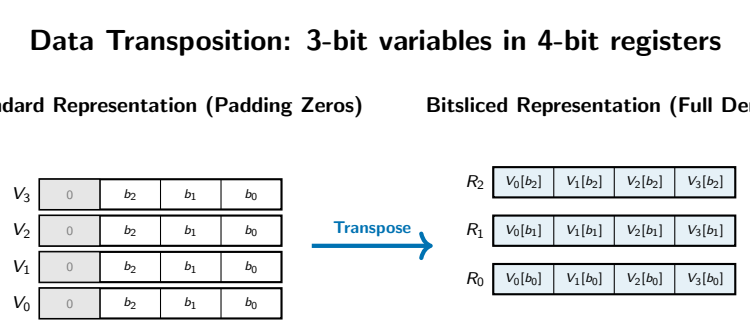
Precompute K_0 and g from p, q, p', q'
repeat
  $Y \leftarrow DL_c(\ln 2)$ `// Discrete Laplace (Alg. 4)`
  $L_Y \leftarrow (2q^2|Yq' - p'| - p^2q')/g$
  $K \leftarrow \mathrm{Geom}(1/2)$ `// Integer part of Exp`
  $D \leftarrow RED_{s,c}$ `// Discrete proxy for fractional part of Exp`
until $K_0K + D \geq L_Y^2$
return Y

Presentation Outline



1. From continuous to discrete
2. Moving everything to integers
3. Efficient gadgets
4. Performance and applications

Implementation Efficiency: Bitslicing



- **Throughput**: Every bit in the register is now part of a useful parallel computation.
- **Enable high-depth circuit**: Only 1-bit gate are evaluated.
- t-**probing security**: Relies on the PINI property.

State-of-the-art PINI Gadgets



State-of-the-art PINI gadgets

- `SecAnd`: The fundamental non-linear operation. Base building block for PINI security.
- `SecLeq`: Secure comparison ($A \leq B$).

State-of-the-art PINI Gadgets



State-of-the-art PINI gadgets

- `SecAnd`: The fundamental non-linear operation. Base building block for PINI security.
- `SecLeq`: Secure comparison ($A \leq B$).

The `SecEq` Gadget (Equality)

- **Goal**: Evaluate $A \stackrel{?}{=} B$ securely.
  1. Compute difference: $D = A \oplus B$ (Linear).
  2. `SecOr` all the bits of D.

Secure Geometric Sampling (SecGeom)



Core Idea: The "Position of the First 1" Trick

The Statistical Principle

For a uniform bitstring u, the index of its **first '1' bit** perfectly follows a Geometric distribution of parameter $p = 1/2$.

Example: $u = 0001\mathbf{1}0110 \implies p = 3$

Secure Geometric Sampling (SecGeom)



Core Idea: The "Position of the First 1" Trick

The Statistical Principle

For a uniform bitstring u, the index of its **first '1' bit** perfectly follows a Geometric distribution of parameter $p = 1/2$.

Example: $u = 000\mathbf{1}0110 \implies p = 3$

The Constant-Time Challenge

We must scan the entire string without branching, hiding which index was selected.

- **Scan**: Read bits from index 0 to $k - 1$.
- `prev` **flag**: Remains 1 as long as we only see '0's.
- **Trigger**: A bit $u[i]$ is the *first* '1' if and only if $u[i] = 1$ AND $prev = 1$.

- **Cost**: The flag update only requires a single non-linear operation per bit of precision.

Simplified SecGeom Algorithm



To extract the index securely, we use a linear `Select(value, condition)` primitive. It returns `value` if `condition == 1`, and 0 otherwise.

Alg. 7 :: `SecGeom` (Simplified Logic)

Input : Precision k
Output: Index $r \sim \text{Geom}(1/2)$ truncated to $k-1$

$u \leftarrow \text{Unif}(\{0,1\}^k)$ // Random bitstring
$prev \leftarrow \neg u[0]$
$r \leftarrow \text{Select}(0, u[0])$ // Initialize accumulator
for $i \in \{1, \ldots, k-1\}$ **do**
  $b \leftarrow prev \wedge u[i]$ // Is it the *first* '1'?
  $r \leftarrow r \oplus \text{Select}(i, b)$ // Add i to output if $b = 1$
  $prev \leftarrow prev \oplus b$ // Clear flag if '1' was found
return r

Simplified SecGeom Algorithm



To extract the index securely, we use a linear `Select(value, condition)` primitive. It returns `value` if `condition == 1`, and 0 otherwise.

Alg. 7 :: `SecGeom` (Simplified Logic)

Input : Target bit-length k
Output: Index $r \sim \text{Geom}(1/2)$ truncated to $k-1$

$u \leftarrow \text{Unif}(\{0,1\}^k)$ // Random bitstring
$prev \leftarrow \neg u[0]$
$r \leftarrow \text{Select}(f(0), u[0])$ // Initialize accumulator
for $i \in \{1, \ldots, k-1\}$ **do**
  $b \leftarrow prev \wedge u[i]$ // Is it the *first* '1'?
  $r \leftarrow r \oplus \text{Select}(f(i), b)$ // Add $f(i)$ to output if $b = 1$
  $prev \leftarrow prev \oplus b$ // Clear flag if '1' was found
return r

Functional SecGeom



Once the "first 1" trigger bit (b) is securely computed, we can use it to evaluate any public function f on the secret index p at **no extra non-linear cost**.

Alg. 8 :: SecGeomX

Input : Bit-length k, public functions $(f_j)_{0 \le j < n}$
Output: Evaluations $r_j = f_j(p)$ where $p \sim \text{Geom}(1/2)$

$u \leftarrow \text{Unif}(\{0,1\}^k)$; $\quad prev \leftarrow \neg u[0]$
for $j \in \{0, \ldots, n-1\}$ **do**
  $r_j \leftarrow \text{Select}(f_j(0), u[0])$
for $i \in \{1, \ldots, k-1\}$ **do**
  $b \leftarrow prev \wedge u[i]$
  for $j \in \{0, \ldots, n-1\}$ **do**
    $r_j \leftarrow r_j \oplus \text{Select}(f_j(i), b)$ `// As many functions as needed`
  $prev \leftarrow prev \oplus b$
return $\{r_j\}_{0 \le j < n}$

From Sampling to Evaluation: SecDLX



SecDLX extends SecGeomX with a branching logic, that enables handling the "sign" of the output.

From Sampling to Evaluation: SecDLX



SecDLX extends SecGeomX with a branching logic, that enables handling the "sign" of the output.

Alg. 8 :: SecDLX (Simplified Logic)

Input : Bit-length k, secret bit S, public functions $(f_{0,j}, f_{1,j})$
Output: Evaluations $r_j = f_{S,j}(p)$

$u \leftarrow \mathsf{Unif}(\{0,1\}^k)$; $\quad prev \leftarrow \neg u[0]$
$\sigma \leftarrow u[0] \wedge S$ // Selection bit for index 0
$r_j \leftarrow \mathsf{Select}(f_{0,j}(0), u[0] \oplus \sigma) \oplus \mathsf{Select}(f_{1,j}(0), \sigma)$
for $i \in \{1, \ldots, k-1\}$ **do**
  $b \leftarrow prev \wedge u[i]$ // First '1' found at index i
  $\sigma \leftarrow b \wedge S$ // Multiplexing: $b \wedge S$ vs $b \wedge \neg S$
  $t_j \leftarrow \mathsf{Select}(f_{0,j}(i), b \oplus \sigma) \oplus \mathsf{Select}(f_{1,j}(i), \sigma)$
  $r_j \leftarrow r_j \oplus t_j$; $\quad prev \leftarrow prev \oplus b$
return $\{r_j\}$

Discrete Laplace sampling



Alg. 4 :: Discrete Laplace $DL_c(\ln 2)$

Input : Center $c \in \mathbb{R}$
Output: $Y \sim DL_c(\ln 2)$

$n \leftarrow \lfloor c \rceil$; $\quad \delta \leftarrow c - n$
$S \leftarrow \text{Bern}\left(\frac{2^\delta}{2^\delta + 2^{1-\delta}}\right)$
$G \leftarrow \text{Geom}(1/2)$
if $S = 1$ **then return** $n + (G + 1)$
else return $n - G$

Final Integer Sampler



Alg. 6 :: Integer Gaussian Sampler

Input : Rational $s = p/q > 0$ and $c = p'/q'$
Output: $Z \sim D_{\mathbb{Z},c,\sigma}$ where $\sigma = s/\sqrt{2\ln 2}$

Precompute K_0 and g from p, q, p', q'
repeat
  $Y \leftarrow DL_c(\ln 2)$ `// Discrete Laplace (Alg. 4)`
  $L_Y \leftarrow (2q^2|Yq' - p'| - p^2q')/g$
  $K \leftarrow \mathrm{Geom}(1/2)$ `// Integer part of Exp`
  $D \leftarrow RED_{s,c}$ `// Discrete proxy for fractional part of Exp`
until $K_0K + D \geq L_Y^2$
return Y

Final Integer Sampler



Alg. 6 :: Integer Gaussian Sampler

Input : Rational $s = p/q > 0$ and $c = p'/q'$
Output: $Z \sim D_{\mathbb{Z},c,\sigma}$ where $\sigma = s/\sqrt{2\ln 2}$

Precompute K_0 and g from p, q, p', q'
repeat
  $Y \leftarrow DL_c(\ln 2)$ `// Discrete Laplace (Alg. 4)`
  $L_Y \leftarrow (2q^2|Yq' - p'| - p^2q')/g$
  $K \leftarrow \mathrm{Geom}(1/2)$ `// Integer part of Exp`
  $D \leftarrow RED_{s,c}$ `// Discrete proxy for fractional part of Exp`
until $K_0K + D \geq L_Y^2$
return Y

Final Integer Sampler



Alg. 6 :: Integer Gaussian Sampler

Input : Rational $s = p/q > 0$ and $c = p'/q'$
Output: $Z \sim D_{\mathbb{Z},c,\sigma}$ where $\sigma = s/\sqrt{2\ln 2}$

Precompute K_0 and g from p, q, p', q'
repeat
  $Y \leftarrow DL_c(\ln 2)$ // Discrete Laplace (Alg. 4)
  $T_Y \leftarrow$ sentinel associated with L_Y
  $K \leftarrow \mathrm{Geom}(1/2)$ // Integer part of Exp
  ~~$D \leftarrow RED_{s,c}$~~ $b_T = U \leq T_Y$
until $K_0 K + D \geq L_Y^2$
return Y

Final Integer Sampler



Alg. 6 :: Integer Gaussian Sampler

Input : Rational $s = p/q > 0$ and $c = p'/q'$
Output: $Z \sim D_{\mathbb{Z},c,\sigma}$ where $\sigma = s/\sqrt{2\ln 2}$

Precompute K_0 and g from p, q, p', q'
repeat
  $Y \leftarrow DL_c(\ln 2)$ // Discrete Laplace (Alg. 4)
  $T_Y \leftarrow$ sentinel associated with L_Y
  $r_Y \leftarrow L_Y \pmod{K_0}$
  $q_Y \leftarrow L_Y / K_0$
  $K \leftarrow \mathrm{Geom}(1/2)$ // Integer part of Exp
  ~~$D \leftarrow RED_{s,c}$~~ $b_T = U \leq T_Y$
until $K > q_Y$ or ($K = q_Y$ and b_T)
return Y

Presentation Outline



1. From continuous to discrete
2. Moving everything to integers
3. Efficient gadgets
4. Performance and applications

SecAnd: The Performance Metric



Non-linear $\implies$ Costly

- **The Bottleneck**: Only non-linear gates require fresh randomness
- **Quadratic Cost**: Execution time grows with $O(d^2)$ for d shares
- **Unit of Measure**: All performance is measured in "number of amortized $\texttt{SecAnd}_{32}$"

Note: Linear operations (XOR, NOT) are considered "free".

State-of-the-Art Comparison: HAWK Signatures



Sampler	HAWK-512		HAWK-1024	
	$c = 0$	$c = 0.5$	$c = 0$	$c = 0.5$
Gérard–Rossi [28]	195 ($\times$ 40.1)	195 ($\times$ 48.8)	195 ($\times$ 40.1)	195 ($\times$ 48.8)
Eid et al. [21]	87 ($\times$ 17.9)	87 ($\times$ 21.7)	87 ($\times$ 17.9)	87 ($\times$ 21.7)
Gérard–Rossi (bitsliced)*	31.7 ($\times$ 6.5)	31.7 ($\times$ 7.9)	31.7 ($\times$ 6.5)	31.7 ($\times$ 7.9)
Eid et al. (bitsliced)*	19.5 ($\times$ 4.0)	19.5 ($\times$ 4.9)	19.5 ($\times$ 4.0)	19.5 ($\times$ 4.9)
Maskaglia (This work)	**4.9**	**4.0**	**4.9**	**4.0**

Various Remarks



- **Scheme-Specific Gains**: Optimizations for $c \in \{0, 1/2\}$, reuse of randomness (HAWK).

Various Remarks



- **Scheme-Specific Gains**: Optimizations for $c \in \{0, 1/2\}$, reuse of randomness (HAWK).
- **Rounded Gaussian**: Adaptable to FrodoKEM distributions with adapted rejection condition.

Various Remarks



- **Scheme-Specific Gains**: Optimizations for $c \in \{0, 1/2\}$, reuse of randomness (HAWK).
- **Rounded Gaussian**: Adaptable to FrodoKEM distributions with adapted rejection condition.
- **Security Boundary**: Not FO-compliant, re-encryption need to follow the same algorithm.

Final boss of implementers: FALCON



Applying Maskaglia to Secret, Variable Parameters

The Paradigm Shift

In HAWK, the standard deviation σ and center c are public, allowing free precomputations (K_0, RED). In FALCON, these parameters are **dynamic and secret**.

Challenge 1: No Precomputations

- Cannot use the discrete *RED* proxy.
- Must evaluate $V \leq 2^{-\{z^2\}}$ dynamically to avoid logarithms.
- Requires masked polynomial approximations.

Challenge 2: Timing Leakage

- Rejection rate depends on the secret σ.
- A standard `repeat-until` loop leaks the key.
- Requires fixed-iteration "dummy" loops (Adaptive Rejection).